

Beatriz São Leandro Cosimatti

17024130

João Victor Bozola Bosi

5979703

João Vitor Vasconcelos Jacob

16987258

Laís Antunes Cayres Quintão

16880392

Trabalho de Estrutura de Dados II:
Análise e Implementação de Algoritmos

Disciplina: Estrutura de Dados II

SCC0606

Professor:

São Carlos
14 jun. 2026

1 Introdução

O objetivo do trabalho desenvolvido foi estudar e implementar diferentes algoritmos de ordenação na linguagem C. Como cada algoritmo se comporta de uma maneira específica dependendo do tamanho do vetor ou de como os números estão organizados, o objetivo final do projeto é a criação de um Sistema Adaptativo. Esse sistema funciona como um analisador que avalia o vetor de entrada e, através de heurísticas, escolhe automaticamente o melhor algoritmo para realizar a ordenação.

Para fundamentar as tomadas de decisão desse sistema, o projeto foi estruturado de maneira modular, englobando a implementação de cinco algoritmos: Bubble Sort, Insertion Sort, Quick Sort, Heap Sort e Radix Sort.

ACHO Q COLOCA O LINK DO VIDEO AQ

2 Algoritmos Base

2.1 Bubble Sort

O Bubble Sort (Ordenação por Bolha) é um algoritmo de ordenação baseado no princípio de comparações e trocas adjacentes. Ele percorre o vetor repetidamente a partir do início comparando pares de elementos vizinhos (vetor[i] e vetor [i+1]), e troca-os de lugar se o elemento da esquerda for maior que o da direita. Esse processo é repetido até que nenhuma troca seja necessária em uma passada completa, indicando que o conjunto de dados está totalmente ordenado, sendo essa a versão aprimorada do Bubble Sort.

Dessa forma, a sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado. Nessa situação, o vetor só é percorrido uma única vez, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes devido ao crescimento quadrático das operações de comparação e movimentação de dados.

2.2 Insertion Sort

O Insertion Sort (Ordenação por Inserção) consiste em inserir um novo elemento em um vetor já ordenado ou quase ordenado. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é inserida. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é finalmente inserida.

Assim, sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado (ou quase ordenado). Nessa situação, a chave é comparada apenas uma vez com o elemento imediatamente anterior e o loop interno é interrompido por um break, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes e aleatórias.

2.3 Quick Sort

O algoritmo de ordenação Quick Sort foi desenvolvido e publicado pela primeira vez em 1961, pelo cientista da computação britânico Charles Antony Richard Hoare (1934 - 2026). A técnica de ordenação foi concebida enquanto o cientista estudava na Universidade de Moscou e trabalhava no *National Physical Laboratory (NPL)*, do Reino Unido. Na ocasião, Hoare estava participando de um projeto que tinha como objetivo a tradução de textos em russo para inglês e, para isso, como os dicionários eram armazenados em fita magnética, necessitava ordenar das palavras dentro de uma sentença em ordem alfabética. Nesse contexto, surgiu a necessidade de desenvolver um algoritmo de ordenação que fosse assintoticamente menor que $O(n^2)$.

2.4 Heap Sort

explicação do Heap

2.5 Radix Sort

explicação do radix

3 Arquitetura do Sistema

Descrição: • das métricas coletadas; • das heurísticas; • das decisões implementadas - a parte da

bia

4 Metodologia Experimental

Descrever: • ambiente utilizado; • datasets; • tamanhos testados; • critérios de avaliação

5 Resultados

Apresentar tabelas, gráficos, métricas coletadas, com discussão crítica. Explicar: • decisões corretas; • decisões ruins; • comportamentos inesperados; • limitações observadas; • impacto das características da entrada. • impactos das entradas adversariais

6 Reflexão sobre Uso de IA

- O grupo deverá informar:
- quais ferramentas de IA foram utilizadas;
 - em quais etapas auxiliaram;
 - erros encontrados nas respostas geradas;
 - correções realizadas pelo grupo.

7 Conclusão

Resumo dos principais achados.

8 Referências Bibliográficas